

COVID-19 STATISTICAL ANALYSIS DASHBOARD

Team Name: Stat-isfaction

#	Roll Number	Name	Section
1	24F-3100	Zubair Azam (Group Leader)	SE (4B)
2	24F-0810	Muhammad Uzair Hussain	CS (4D)
3	24F-0512	Usman Ali	CS (4D)
4	24F-0567	Taheer Bin Hussain	CS (4D)
5	24F-3095	Usman Haider	SE (4B)

Table of Contents

1 Problem Statement	1
2 Objective	2
3 Data Description	4
3.1 Dataset Source & Overview	4
3.2 Variable Groups	6
3.3 Data Pre-processing Pipeline	8
4 Results	10
4.1 Tab 1: Global Overview	10
4.2 Tab 2: Time Series Analysis	11
4.3 Tab 3: Descriptive Statistics	12
4.4 Tab 4: Probability & Distribution Analysis	14
4.5 Tab 5: Regression Analysis	15
4.6 Tab 6: Country Comparison	17
5 Code	19
6 Conclusion	33
7 References	35

1. Problem Statement

The COVID-19 pandemic, declared by the World Health Organization (WHO) in March 2020, became one of the most catastrophic global health crises in modern history. Within two years, the virus infected more than 600 million people across 200+ countries and claimed over 6.5 million lives. Governments, public health authorities, and researchers scrambled to understand how the disease spread, which populations were most vulnerable, how effective government interventions were, and how vaccination campaigns impacted outcomes.

While vast quantities of epidemiological, socio-economic, and demographic data were collected during this period (most notably by Our World in Data (OWID), Johns Hopkins University, and the Oxford COVID-19 Government Response Tracker), this data in its raw form is largely inaccessible to non-specialists and difficult to analyze without computational tools.

The core problem this project addresses is threefold:

- Lack of integration: COVID-19 data is spread across dozens of columns covering cases, deaths, vaccinations, testing, government stringency, and socio-economic indicators. No single interactive tool applies formal statistical methods to this integrated dataset.
- Absence of statistical rigor: Most dashboards (e.g., Johns Hopkins tracker) present raw counts only, and they do not compute confidence intervals, fit probability distributions, test statistical hypotheses, or run regression models that reveal causal or predictive relationships.
- Accessibility barrier: Raw CSV data and Python scripts are not accessible to policymakers, students, or the general public. An interactive web interface is required to make statistical insights explorable without programming knowledge.

In summary, this project solves the problem of transforming a large, complex, multi-country pandemic dataset into a statistically rigorous, visually rich, and interactively explorable analytical dashboard, grounding all visualizations in the formal methods of probability and statistics taught in this course.

2. Objective

The project pursues the following specific, measurable objectives, each directly mapping to a module of the dashboard:

- **Obj 1: Data Engineering:**
Load the OWID COVID-19 CSV dataset (200,000 + rows, 67 columns), parse dates, filter to real countries (removing aggregate rows like 'World' and 'High income'), and produce clean per-country snapshot statistics via grouped aggregation, forming the foundation for all subsequent analyses.
- **Obj 2: Global Overview:**
Generate a world choropleth map, top 15 country bar chart, continent-share donut chart, and a log-log scatter plot of total deaths versus total cases. Compute and display six global KPIs: total cases, total deaths, total vaccinations, case fatality rate (CFR), number of countries tracked, and data coverage period.
- **Obj 3: Time-Series Exploration:**
Enable interactive multi-country, multi-metric time-series visualisation for any of six metrics (new cases, total cases, new deaths, total deaths, new vaccinations, stringency index), including a 7-day rolling average chart to remove weekly reporting artefacts.
- **Obj 4: Descriptive Statistics:**
Compute and display a complete describe() summary table, compute 95% confidence intervals using the Student's t-distribution (`scipy.stats.t.interval`), and render box plots and histograms for key variables.
- **Obj 5: Probability & Distribution Analysis:**
Fit Normal and Log-Normal distributions to selected variables; generate overlaid histogram and PDF plots; produce Q-Q plots; run Shapiro-Wilk ($n \leq 5000$) or Kolmogorov-Smirnov ($n > 5000$) normality tests; report skewness, kurtosis, and empirical rule coverage; provide an interactive probability calculator for $P(X > T)$ and $P(X < T)$.
- **Obj 6: Regression Analysis:**
Perform simple OLS linear regression between any user-selected X–Y pair; display scatter plot with regression line, summary statistics (slope, intercept,

R^2 , r , p -value, SE), regression equation, significance verdict, and a residuals vs fitted values plot. Additionally fit a multiple regression model (4 predictors $\rightarrow$ total deaths) using `statsmodels.OLS` and display coefficient table with t -statistics and p -values. Provide a real-time prediction tool with 95% prediction intervals.

- **Obj 7: Country Comparison:**

Allow multi-country side-by-side comparison via a normalised radar (spider) chart, grouped bar charts (cases, deaths, vaccinations), and a summary comparison table with CFR%, cases/million, deaths/million, life expectancy, GDP per capita, and mean stringency index.

3. Data Description

3.1. Dataset Source & Overview

The dataset used is the Our World in Data (OWID) COVID-19 Complete Dataset, one of the most comprehensive, openly available, and frequently updated repositories of pandemic data. It consolidates information from multiple authoritative sources including the World Health Organization (WHO), the European Centre for Disease Prevention and Control (ECDC), the Oxford COVID-19 Government Response Tracker (OxCGRT), and various national health ministries.

File:

covid_data.csv

Source URL:

<https://github.com/owid/covid-19-data/tree/master/public/data>

Maintainer:

Our World in Data (Hannah Ritchie, Edouard Mathieu et al.)

License:

Creative Commons BY 4.0 (free to use with attribution)

Attribute	Detail
Total Rows (observations)	~200,000+ (one row per country per date)
Total Columns (features)	67 variables spanning epidemiology, vaccination, policy, demographics
Countries / Territories	200+ locations (after filtering out aggregate rows)
Date Range	January 2020 – December 2022 (3 years)
Granularity	Daily (each row = one country on one calendar date)
Missing Values	Extensive (many countries lack testing, vaccination, or HDI data)
File Size (approx.)	~80 MB uncompressed

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	iso_code	continent	location	date	total_cases	new_cases	new_cases_smoothed	total_deaths	new_deaths	new_deaths_smoothed	total_cases_per_million	new_cases_per_million	total_deaths_per_million	reproduction_rate	hosp_patients	total_tests
2	AFG	Asia	Afghanistan	24-02-2020	5	5	0	0	0	0	0.125	0.125	0	0	0	0
3	AFG	Asia	Afghanistan	25-02-2020	5	0	0	0	0	0	0.125	0	0	0	0	0
4	AFG	Asia	Afghanistan	26-02-2020	5	0	0	0	0	0	0.125	0	0	0	0	0
5	AFG	Asia	Afghanistan	27-02-2020	5	0	0	0	0	0	0.125	0	0	0	0	0
6	AFG	Asia	Afghanistan	28-02-2020	5	0	0	0	0	0	0.125	0	0	0	0	0
7	AFG	Asia	Afghanistan	29-02-2020	5	0	0.714	0	0	0	0.125	0	0	0	0	0
8	AFG	Asia	Afghanistan	01-03-2020	5	0	0.714	0	0	0	0.125	0	0	0	0	0
9	AFG	Asia	Afghanistan	02-03-2020	5	0	0	0	0	0	0.125	0	0	0	0	0
10	AFG	Asia	Afghanistan	03-03-2020	5	0	0	0	0	0	0.125	0	0	0	0	0
11	AFG	Asia	Afghanistan	04-03-2020	5	0	0	0	0	0	0.125	0	0	0	0	0
12	AFG	Asia	Afghanistan	05-03-2020	5	0	0	0	0	0	0.125	0	0	0	0	0
13	AFG	Asia	Afghanistan	06-03-2020	5	0	0	0	0	0	0.125	0	0	0	0	0
14	AFG	Asia	Afghanistan	07-03-2020	8	3	0.429	0	0	0	0.2	0.075	0	0	0	0
15	AFG	Asia	Afghanistan	08-03-2020	8	0	0.429	0	0	0	0.2	0	0	0	0	0
16	AFG	Asia	Afghanistan	09-03-2020	8	0	0.429	0	0	0	0.2	0	0	0	0	0
17	AFG	Asia	Afghanistan	10-03-2020	8	0	0.429	0	0	0	0.2	0	0	0	0	0
18	AFG	Asia	Afghanistan	11-03-2020	11	3	0.857	0	0	0	0.274	0.075	0	0	0	0
19	AFG	Asia	Afghanistan	12-03-2020	11	0	0.857	0	0	0	0.274	0	0	0	0	0
20	AFG	Asia	Afghanistan	13-03-2020	11	0	0.857	0	0	0	0.274	0	0	0	0	0
21	AFG	Asia	Afghanistan	14-03-2020	14	3	0.857	0	0	0	0.349	0.075	0	0	0	0
22	AFG	Asia	Afghanistan	15-03-2020	20	6	1.714	0	0	0	0.499	0.15	0	0	0	0
23	AFG	Asia	Afghanistan	16-03-2020	25	5	2.429	0	0	0	0.623	0.125	0	0	0	0
24	AFG	Asia	Afghanistan	17-03-2020	26	1	2.571	0	0	0	0.648	0.025	0	0	0	0
25	AFG	Asia	Afghanistan	18-03-2020	26	0	2.143	0	0	0	0.648	0	0	0	0	0
26	AFG	Asia	Afghanistan	19-03-2020	26	0	2.143	0	0	0	0.648	0	0	0	0	0
27	AFG	Asia	Afghanistan	20-03-2020	24	0	0	0	0	0	0.599	0	0	0	0	0
28	AFG	Asia	Afghanistan	21-03-2020	24	0	1.714	0	0	0	0.599	0	0	0	0	0
29	AFG	Asia	Afghanistan	22-03-2020	34	10	2.286	0	0	0	0.848	0.249	0	0	0	0
30	AFG	Asia	Afghanistan	23-03-2020	40	6	2.429	1	1	0	0.998	0.15	0.025	0	0	0
31	AFG	Asia	Afghanistan	24-03-2020	42	2	2.571	1	0	0	1.047	0.05	0.025	0	0	0
32	AFG	Asia	Afghanistan	25-03-2020	74	32	7.143	1	0	0	1.845	0.798	0.025	0	0	0
33	AFG	Asia	Afghanistan	26-03-2020	80	6	8	2	1	0	1.995	0.15	0.05	0	0	0
34	AFG	Asia	Afghanistan	27-03-2020	91	11	9.571	2	0	0	2.269	0.274	0.05	0	0	0
35	AFG	Asia	Afghanistan	28-03-2020	106	15	11.714	4	0	0.286	2.843	0.274	0.05	0	0	0
36	AFG	Asia	Afghanistan	29-03-2020	114	8	11.429	4	2	0.571	2.843	0.2	0.1	1.51	0	0
37	AFG	Asia	Afghanistan	30-03-2020	114	0	10.571	4	0	0.429	2.843	0	0.1	1.51	0	0
38	AFG	Asia	Afghanistan	31-03-2020	166	52	17.714	4	0	0.429	4.14	1.297	0.1	1.52	0	0
39	AFG	Asia	Afghanistan	01-04-2020	192	26	16.857	4	0	0.429	4.788	0.648	0.1	1.51	0	0
40	AFG	Asia	Afghanistan	02-04-2020	235	43	22.143	4	0	0.286	5.86	1.072	0.1	1.51	0	0
41	AFG	Asia	Afghanistan	03-04-2020	269	34	25.429	5	1	0.429	6.708	0.848	0.125	1.5	0	0
42	AFG	Asia	Afghanistan	04-04-2020	270	1	23.429	5	0	0.429	6.723	0.025	0.125	1.49	0	0
43	AFG	Asia	Afghanistan	05-04-2020	299	29	26.429	7	2	0.429	7.456	0.723	0.175	1.49	0	0
44	AFG	Asia	Afghanistan	06-04-2020	337	38	31.857	7	0	0.429	8.404	0.948	0.175	1.49	0	0
45	AFG	Asia	Afghanistan	07-04-2020	367	30	28.714	11	4	1	9.152	0.748	0.274	1.49	0	0
46	AFG	Asia	Afghanistan	08-04-2020	423	56	33	14	3	1.429	10.549	1.397	0.349	1.49	0	0

Figure 1 : Dataset Preview: Columns A-P (Identification, Cases, Deaths)

	Q	R	S	T	U	V	W	X	Y	Z	AA	AB	AC	AD	AE	AF
1	positive_rate	tests_units	total_vaccinations	people_vaccinated	people_fully_vaccinated	total_boosters	new_vaccinations	stringency_index	population	population_density	aged_65_plus	aged_70_plus	gdp_per_capita	cardiovascular_death_rate	diabetes_prevalence	hospital_beds_per
2	0	0	0	0	0	0	0	8.33	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
3	0	0	0	0	0	0	0	8.33	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
4	0	0	0	0	0	0	0	8.33	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
5	0	0	0	0	0	0	0	8.33	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
6	0	0	0	0	0	0	0	8.33	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
7	0	0	0	0	0	0	0	8.33	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
8	0	0	0	0	0	0	0	27.78	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
9	0	0	0	0	0	0	0	27.78	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
10	0	0	0	0	0	0	0	27.78	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
11	0	0	0	0	0	0	0	27.78	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
12	0	0	0	0	0	0	0	27.78	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
13	0	0	0	0	0	0	0	27.78	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
14	0	0	0	0	0	0	0	27.78	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
15	0	0	0	0	0	0	0	27.78	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
16	0	0	0	0	0	0	0	27.78	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
17	0	0	0	0	0	0	0	27.78	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
18	0	0	0	0	0	0	0	27.78	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
19	0	0	0	0	0	0	0	27.78	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
20	0	0	0	0	0	0	0	27.78	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
21	0	0	0	0	0	0	0	38.89	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
22	0	0	0	0	0	0	0	38.89	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
23	0	0	0	0	0	0	0	38.89	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
24	0	0	0	0	0	0	0	38.89	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
25	0	0	0	0	0	0	0	41.67	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
26	0	0	0	0	0	0	0	41.67	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
27	0	0	0	0	0	0	0	41.67	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
28	0	0	0	0	0	0	0	41.67	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
29	0	0	0	0	0	0	0	41.67	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
30	0	0	0	0	0	0	0	41.67	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
31	0	0	0	0	0	0	0	50.93	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
32	0	0	0	0	0	0	0	59.26	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
33	0	0	0	0	0	0	0	59.26	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
34	0	0	0	0	0	0	0	59.26	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
35	0	0	0	0	0	0	0	67.59	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
36	0	0	0	0	0	0	0	67.59	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
37	0	0	0	0	0	0	0	67.59	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
38	0	0	0	0	0	0	0	67.59	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
39	0	0	0	0	0	0	0	67.59	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
40	0	0	0	0	0	0	0	67.59	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
41	0	0	0	0	0	0	0	67.59	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
42	0	0	0	0	0	0	0	67.59	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
43	0	0	0	0	0	0	0	78.7	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
44	0	0	0	0	0	0	0	78.7	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
45	0	0	0	0	0	0	0	78.7	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59
46	0	0	0	0	0	0	0	78.7	40099462	54.422	2.581	1.337	1803.987	597.029	9.59	9.59

	Z	AA	AB	AC	AD	AE	AF	AG	AH
1	population_density	aged_65_older	aged_70_older	gdp_per_capita	cardiovasc_death_rate	diabetes_prevalence	hospital_beds_per_thousand	life_expectancy	human_development_index
2	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
3	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
4	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
5	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
6	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
7	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
8	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
9	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
10	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
11	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
12	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
13	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
14	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
15	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
16	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
17	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
18	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
19	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
20	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
21	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
22	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
23	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
24	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
25	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
26	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
27	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
28	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
29	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
30	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
31	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
32	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
33	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
34	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
35	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
36	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
37	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
38	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
39	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
40	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
41	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
42	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
43	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
44	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
45	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511
46	54.422	2.581	1.337	1803.987	597.029	9.59	0.5	64.83	0.511

Figure 3 : Dataset Preview: Columns Z-AH (Socio-Economic Indicators)

3.2. Variable Groups

The 67 columns are organized into five thematic groups:

- **Identification**
 - **Variables:** *iso_code, continent, location, date*
 - **Description:** Uniquely identifies each row: ISO-3 country code, continent label, country name, date.
- **Epidemiological: Cases**
 - **Variables:** *total_cases, new_cases, new_cases_smoothed, total_cases_per_million, new_cases_per_million, new_cases_smoothed_per_million, reproduction_rate*
 - **Description:** Cumulative and daily case counts (raw and per-million), 7-day smoothed series, and the estimated reproduction number R.

- **Epidemiological: Deaths & Hospital**
 - **Variables:** *total_deaths, new_deaths, new_deaths_smoothed, total_deaths_per_million, new_deaths_per_million, new_deaths_smoothed_per_million, hosp_patients, hosp_patients_per_million, weekly_hosp_admissions, icu_patients*
 - **Description:** Mortality counts (raw and per-million), hospital occupancy, and ICU load.
- **Testing**
 - **Variables:** *total_tests, new_tests, total_tests_per_thousand, new_tests_per_thousand, positive_rate, tests_per_case, tests_units*
 - **Description:** Volume of diagnostic tests conducted; positivity rate = $\text{new_cases} / \text{new_tests}$; tests_units documents whether counts are PCR, antigen, or combined.
- **Vaccination**
 - **Variables:** *total_vaccinations, people_vaccinated, people_fully_vaccinated, total_boosters, new_vaccinations, new_vaccinations_smoothed, total_vaccinations_per_hundred, people_vaccinated_per_hundred, people_fully_vaccinated_per_hundred*
 - **Description:** Cumulative and daily vaccination doses; percentage coverage of the population.
- **Government Policy**
 - **Variables:** *stringency_index*
 - **Description:** Oxford OxCGR Stringency Index (0–100). Higher values indicate stricter lockdowns, school closures, travel bans, and gathering restrictions.
- **Socio-Economic & Demographic**
 - **Variables:** *population, population_density, median_age, aged_65_older, aged_70_older, gdp_per_capita, extreme_poverty, cardiovasc_death_rate, diabetes_prevalence, female_smokers, male_smokers, handwashing_facilities,*

*hospital_beds_per_thousand, life_expectancy,
human_development_index*

- **Description:** Country-level static indicators from World Bank, UN HDR, and WHO that serve as predictors in regression models.

3.3. Data Pre-processing Pipeline

Raw data required several cleaning and transformation steps before analysis. These are applied in the following order at application startup:

- **Step 1: Date Parsing**
`df_raw['date'] = pd.to_datetime(df_raw['date'], dayfirst=True)`
converts the string date column into proper datetime 64 objects required for time-series plots and chronological sorting.
- **Step 2: Country Filtering**
`df = df_raw[df_raw['continent'].notna()].copy()`
removes 'aggregate' rows such as 'World', 'Asia', 'Europe', 'High income', 'Low income', etc., which have NaN in the continent column. This ensures all analyses reflect individual countries only.
- **Step 3: Chronological Sorting**
`df = df.sort_values(['location', 'date'])`
sorts by country first, then by date, enabling correct rolling-window calculations and time-series rendering.
- **Step 4: Country / Continent Lists**
`countries = sorted(df['location'].unique().tolist())`
builds the dropdown menus. These lists are computed once and reused across all tab callbacks.
- **Step 5: Global KPI Aggregation**
For the KPI strip, `df.groupby('location').agg(max)` is applied once at startup to avoid redundant computation on every page load. This gives the peak cumulative total for cases, deaths, and vaccinations per country before summing globally.

- **Step 6: NaN / Inf Handling**

Inside each callback, `.dropna(subset=[cols])` and `.replace([np.inf, -np.inf], np.nan)` followed by `.dropna()` ensure that regression and distribution functions never receive invalid floating-point values.

- **Step 7: Outlier Clipping (Histogram)**

`new_cases` is clipped at the 95th percentile (`snap['new_cases'].quantile(0.95)`) before histogram rendering to prevent a few extreme outliers from compressing the rest of the distribution into a single bin.

4. Results

The dashboard is a Dash/Plotly web application running on localhost:8050. It consists of a persistent header, a six-metric KPI strip, and six interactive tabs. Every chart is fully interactive (hover, zoom, pan, legend toggle). The following subsections describe every result produced, as observed in the web interface.

4.1. Tab 1: Global Overview

This tab provides a high-level, country- and continent-level snapshot of the pandemic. It is the default landing tab and contains four charts plus the KPI strip.

KPI Strip (Persistent across all tabs)

KPI	Value	Computation
Total Cases	≈ 620 Million	Sum of per-country peak total cases
Total Deaths	≈ 6.56 Million	Sum of per-country peak total deaths
Total Vaccinations	≈ 12.84 Billion	Sum of per-country peak total vaccinations
Case Fatality Rate	$\approx 1.06\%$	Global deaths / Global cases $\times 100$
Countries Tracked	221	<code>df['location'].nunique()</code>
Data Period	2020 – 2022	Fixed label

World Choropleth Map

A filled world map generated by `px.choropleth()` coloured on a Blues colour scale proportional to total confirmed cases. The United States (darkest), India, and Brazil dominate. Sub-Saharan Africa shows lighter colouring, reflecting lower confirmed case counts (though this may be partly attributable to under-testing rather than lower actual infection rates). The map is interactive: hovering over any country displays its name and exact case count in a tooltip.

Top-15 Countries by Total Cases: Horizontal Bar Chart

A horizontal bar chart produced by `px.bar()` with `orientation='h'`, sorted ascending (`yaxis_categoryorder='total ascending'`) so the highest-burden country appears at the top. The top five countries (USA, India, France, Germany, Brazil) collectively account for more than 40% of all reported global cases.

Continent Share of Cases: Donut Chart

A `px.pie()` with `hole=0.45` showing each continent's cumulative share of `new_cases` summed over the entire data period. Europe and Asia account for the largest shares (~30% each), followed by North America (~22%). Africa's share (~4%) underscores the testing-gap problem.

Total Deaths vs Total Cases: Log-Log Scatter Plot

A `px.scatter()` on log-log axes with bubble size proportional to total deaths and colour coded by continent. The near-linear log-log relationship implies that deaths scale as a power law with cases, consistent with a roughly constant CFR across orders of magnitude of case counts. Outliers above the trend line (e.g. some Eastern European countries) indicate elevated CFR due to healthcare capacity constraints or older populations.

4.2. Tab 2: Time Series Analysis

The time-series tab provides two linked charts for any user-selected combination of countries and one of six metrics.

Raw Daily Line Chart

`go.Scatter()` traces one line per country. Hover mode is set to 'x unified', meaning hovering over any date shows all selected countries' values in a single tooltip panel. Key observations:

- **United States:**
Three clear waves: April 2020, January 2021 (peak ~300,000 new cases/day), and January 2022 (Omicron surge, highest peak globally).
- **India:**
One dominant Delta wave in May 2021 (~400,000 new cases/day).
- **Pakistan:**
Lower absolute case counts but visible waves correlated with global surges.
- **Germany:**
Multiple symmetric waves with characteristic 'Christmas dip' due to reduced testing.

- **Brazil:**

Persistent high baseline with peaks in March 2021 and January 2022.

7-Day Rolling Average Chart

`d['rolling'] = d[metric].rolling(7, min_periods=1).mean()` computes the moving average. `min_periods=1` ensures the first 6 days are not blank, they are averaged over fewer days. The rolling chart eliminates the sharp weekly sawtooth pattern (caused by under-reporting on weekends) and reveals smooth epidemic curves that align with published epidemiological analyses. This is the format preferred by public health agencies.

4.3. Tab 3: Descriptive Statistics

Descriptive Summary Table

`pandas .describe().T` is computed on a per-country snapshot (max value per country) for ten key variables. Selected results from the dashboard:

Variable	N	Mean	Std Dev	Min	25%	Median	75%	Max
new_cases	221	1,295,682	3,284,401	0	7,011	188,682	776,491	31,335,722
total_deaths	221	29,659	83,209	0	248	3,208	24,082	1,087,938
positive_rate	186	0.107	0.094	0.001	0.038	0.082	0.151	0.615
life_expectancy	196	73.5	7.5	52.3	67.9	74.9	79.2	86.8
gdp_per_capita	183	19,680	20,445	661	4,073	12,951	30,450	116,935
stringency_index (avg)	221	43.2	17.1	5.8	30.6	44.2	56.1	87.4

95% Confidence Intervals

Computed using `scipy.stats.t.interval(0.95, df=n-1, loc=mean, scale=sem)` where `sem = scipy.stats.sem(vals) = std / sqrt(n)`. The t-distribution (not z) is used because the number of countries ($n \approx 180-221$) is moderate rather than truly infinite. An error-bar chart visualises each CI.

Interpretation: if we were to resample 100 independent datasets of the same size, approximately 95 of the resulting intervals would contain the true global mean.

Variable	N	Mean	Std Error	95% CI Lower	95% CI Upper
New Cases	221	1,295,682	220,779	857,854	1,733,510
Total Deaths	221	29,659	5,583	18,671	40,647
Positive Rate	186	0.107	0.007	0.093	0.121
Life Expectancy	196	73.5	0.535	72.4	74.5
GDP per Capita	183	19,680	1,512	16,700	22,660

Box Plots

Side-by-side box plots (go.Box with boxmean='sd') are rendered for new_cases, new_deaths, positive_rate, and life_expectancy. Key observations:

- **New_cases and New_deaths:**
Extreme right-skew with many outlier dots far above the upper whisker (a hallmark of power-law distributed phenomena).
- **Positive_rate:**
Moderate right-skew; the median (~8%) sits below the mean (~10.7%), indicating that a minority of countries drive the average up.
- **Life_expectancy:**
Approximately symmetric, near-normal distribution centred around 75 years with a few low-income country outliers below 60.

Histogram: Distribution of New Cases

A 40-bin histogram of new_cases clipped at the 95th percentile. The resulting distribution is heavily right-skewed (L-shaped), confirming that the majority of countries reported fewer than 500,000 peak daily new cases, while a handful of large countries (USA, India, France) account for the far right tail. This motivates the use of log-normal distribution fitting in the Probability tab.

4.4. Tab 4: Probability & Distribution Analysis

For each selectable variable (new_cases, total_deaths, positive_rate, life_expectancy, gdp_per_capita), the dashboard performs a full probabilistic analysis.

Histogram with PDF Overlay

The observed data histogram is plotted alongside the fitted Normal PDF curve. A second Log-Normal curve is also overlaid for comparison. Parameters are fitted via Maximum Likelihood Estimation using `scipy.stats.norm.fit()` and `scipy.stats.lognorm.fit()`.

Q-Q Plot (Quantile-Quantile Plot)

`scipy.stats.probplot()` generates theoretical normal quantiles on the x-axis and sample quantiles on the y-axis. A diagonal line represents perfect normality. Deviations at the tails (S-curve or heavy tail) indicate skewness or kurtosis. For new_cases, the Q-Q plot shows strong upper-tail deviation, confirming non-normality.

Normality Testing

Variable	Test Used	p-value	Verdict	Skewness
new_cases	Shapiro-Wilk	< 0.001	NOT normal (Log-Normal recommended)	~4.2
total_deaths	Shapiro-Wilk	< 0.001	NOT normal (Log-Normal recommended)	~3.8
positive_rate	Shapiro-Wilk	0.003	NOT normal (moderate right-skew)	~1.6
life_expectancy	Shapiro-Wilk	0.041	Borderline (approximately normal)	~-0.5
gdp_per_capita	Shapiro-Wilk	< 0.001	NOT normal (strong right-skew)	~2.1

Empirical Rule Analysis

For each variable the dashboard reports what percentage of countries fall within 1σ and 2σ of the mean. For a perfect Normal distribution theory predicts 68.27% and 95.45% respectively. Example for life_expectancy

(most normal of the five): within $1\sigma \approx 69\%$, within $2\sigma \approx 94\%$, close to theoretical predictions, consistent with near-normality.

Interactive Probability Calculator

User inputs a threshold T . The system computes:

- $P(X > T)$ using $1 - \text{scipy.stats.norm.cdf}(T, \mu, \sigma)$ (model-based).
- $P(X < T)$ using $\text{scipy.stats.norm.cdf}(T, \mu, \sigma)$ (model-based).
- Empirical $P(X > T) = (\text{vals} > T).\text{mean}()$ (directly from the data, no distribution assumed).
- Count of countries above threshold: n_{above} out of n_{total} .

Example:

For `total_deaths` with $T = 10,000$, model $P(X > 10,000) \approx 0.62$ (62%), empirical ≈ 0.58 (58%). The small discrepancy reflects the non-normality of the distribution.

4.5. Tab 5: Regression Analysis

Simple Linear Regression

User selects any predictor X from `{gdp_per_capita, life_expectancy, stringency_index, aged_65_older, hospital_beds_per_thousand, diabetes_prevalence}` and any target Y from `{total_deaths, total_cases, positive_rate}`. OLS is fitted via `scipy.stats.linregress()`.

Key results for the default configuration ($X = \text{GDP per Capita}$, $Y = \text{Total Deaths}$):

Statistic	Value	Interpretation
Slope (β_1)	3.84	Each \$1,000 increase in GDP/capita is associated with 3,840 more reported deaths (higher-income = better detection)
Intercept (β_0)	-42,113	Predicted deaths when GDP = 0 (extrapolation, not meaningful)
R^2 (Coeff. Det.)	0.2741	GDP per capita explains 27.4% of variance in total deaths
r (Pearson Corr.)	0.5236	Moderate positive correlation
p-value	< 0.001	Statistically significant at $\alpha = 0.05$
Std Error of Slope	0.5102	Precision of the slope estimate

Prediction Tool with 95% Prediction Interval

Given a user-entered X value, the model computes:

- Point prediction: $\hat{Y} = \beta_1 \cdot X + \beta_0$
- Standard error of prediction: $SE_{pred} = s_e \cdot \sqrt{(1 + 1/n + (X - \bar{X})^2 / \sum(X_i - \bar{X})^2)}$
- t-critical value at $\alpha/2 = 0.025$ with $(n-2)$ degrees of freedom: $t^* = \text{stats.t.ppf}(0.975, n-2)$
- 95% PI: $[\hat{Y} - t^* \cdot SE_{pred}, \hat{Y} + t^* \cdot SE_{pred}]$

Example:

For GDP per capita = 30,000 $\rightarrow$ Predicted total deaths $\approx 72,887$;

95% PI: $[-114,400, 260,174]$. The wide interval reflects high country-to-country variability not explained by GDP alone.

Residuals vs Fitted Values Plot

Residuals ($Y_i - \hat{Y}_i$) plotted against fitted values ($\hat{Y}_i$). For an ideal model, residuals should be randomly scattered around zero with no pattern (homoscedasticity). The actual plot shows a funnel shape (residuals increase with fitted values), indicating heteroscedasticity. This is expected given the right-skewed, non-normal distribution of total deaths, and suggests that a log-transformed regression would be more appropriate.

Multiple Linear Regression

A fixed model is fitted: Total Deaths $\sim$ GDP per Capita + Life Expectancy + %Aged 65+ + Hospital Beds per Thousand. Fitted using statsmodels.OLS with sm.add_constant() to include the intercept.

Predictor	Coefficient	Std Error	t-stat	p-value	Significant?
Intercept	-1,345,200	63,810	-21.08	< 0.001	Yes
GDP per Capita	+2.91	0.67	+4.34	< 0.001	Yes
Life Expectancy	+16,820	845	+19.91	< 0.001	Yes
% Aged 65+	+3,012	2,140	+1.41	0.161	No
Hospital Beds per Thousand	-4,388	3,720	-1.18	0.240	No

Model $R^2 \approx 0.352$, Adjusted $R^2 \approx 0.338$, F-statistic = 26.4, Prob(F) < 0.001. Interpretation: the model is globally highly significant (F-test $p < 0.001$). GDP per Capita and Life Expectancy are individually significant ($p < 0.05$), while % Aged 65+ and Hospital Beds per Thousand are not (likely due to multicollinearity with Life Expectancy).

4.6. Tab 6: Country Comparison

The comparison tab allows selection of any set of countries for head-to-head analysis. Default selection: Pakistan, India, United States, Germany, Brazil.

Radar (Spider) Chart

Five metrics (CFR (%), Cases/Million, Deaths/Million, Life Expectancy, and Mean Stringency Index) are normalised to a 0–100 scale (each country's value divided by the maximum across selected countries, then multiplied by 100). `go.Scatterpolar()` draws each country as a closed polygon. Key observation for default countries: Germany scores highest on Cases/Million; Pakistan scores lowest on Life Expectancy and Cases/Million (reflecting under-detection); the USA has the highest absolute deaths but a lower CFR than Pakistan.

Grouped Bar Charts

`make_subplots(rows=1, cols=3)` produces three side-by-side bar charts comparing Total Cases, Total Deaths, and Total Vaccinations across selected countries. Colour-coded by country for easy cross-panel matching.

Comparison Summary Table

Country	Total Cases	Total Deaths	CFR%	Cases/M	Deaths/M	Life Exp.	GDP/Cap	Avg Stringency
Pakistan	1,575,422	30,632	1.94%	7,129	138	67.3	5,839	47.1
India	44,690,468	530,779	1.19%	32,399	384	69.7	6,427	58.6
United States	99,034,531	1,087,938	1.10%	297,576	3,267	78.5	63,528	57.9
Germany	36,441,973	160,013	0.44%	434,578	1,908	81.3	50,801	57.1
Brazil	36,399,477	695,703	1.91%	171,146	3,270	75.9	14,103	52.1

Key comparative insight: Germany has 12× more cases per million than Pakistan but only 14× more deaths, suggesting more comprehensive testing in Germany. The USA has the highest absolute case and death counts but a relatively low CFR (1.10%), reflecting high testing rates and healthcare access. Brazil's CFR (1.91%)

rivals Pakistan's despite a much higher GDP per capita, reflecting healthcare system strain during peak waves.

5. Code

The complete Python source code is presented below in annotated sections. Each section corresponds to a logical module of the application. The application runs as a Dash web server on <http://localhost:8050>.

Libraries required: pandas, numpy, plotly, dash, scipy, statsmodels. Install with:
`pip install pandas numpy plotly dash scipy statsmodels`

```
#  
  
=====
```

```
# SECTION 1: IMPORTS  
#  
=====
```

```
import pandas as pd          # Data manipulation  
import numpy as np           # Numerical operations  
import plotly.graph_objects as go  # Low-level Plotly charts  
import plotly.express as px      # High-level Plotly charts  
from plotly.subplots import make_subplots # Multi-panel figures  
import scipy.stats as stats      # Statistical functions  
import statsmodels.api as sm     # OLS regression  
from dash import Dash, dcc, html, Input, Output, State, dash_table  
import warnings  
warnings.filterwarnings('ignore') # Suppress convergence warnings
```

```
#  
  
=====
```

```
# SECTION 2: LOAD & CLEAN DATA  
#  
=====
```

```
df_raw = pd.read_csv('covid_data.csv')  
df_raw['date'] = pd.to_datetime(df_raw['date'], dayfirst=True)  
  
# Keep only real countries, drop aggregates (continent == NaN)  
df = df_raw[df_raw['continent'].notna()].copy()
```

```

df = df.sort_values(['location', 'date']) # Chronological order

countries = sorted(df['location'].unique().tolist())
continents = sorted(df['continent'].dropna().unique().tolist())

#
=====
=====

# SECTION 3: COLOUR PALETTE & PLOT DEFAULTS
#
=====
=====

COLORS = {
    'bg': '#f0f4f8',    'card': '#ffffff',
    'card2': '#f8fafc', 'accent1': '#1d6fa4', # Blue
    'accent2': '#dc2626', 'accent3': '#d97706', # Red, Amber
    'accent4': '#16a34a', 'accent5': '#7c3aed', # Green, Purple
    'text': '#1e293b',  'muted': '#64748b',
    'border': '#e2e8f0',
}

CONT_COLORS = { # One colour per continent
    'Asia': '#1d6fa4',    'Europe': '#7c3aed',
    'North America': '#d97706', 'South America': '#16a34a',
    'Africa': '#dc2626',    'Oceania': '#f97316',
}

PLOT_LAYOUT = dict(
    paper_bgcolor='rgba(0,0,0,0)',
    plot_bgcolor='rgba(248,250,252,0.9)',
    font=dict(color=COLORS['text'], size=12),
    xaxis=dict(gridcolor=COLORS['border'], linecolor=COLORS['border']),
    yaxis=dict(gridcolor=COLORS['border'], linecolor=COLORS['border']),
    legend=dict(bgcolor='rgba(255,255,255,0.8)', bordercolor=COLORS['border']),
    margin=dict(l=50, r=20, t=50, b=50),
)

```

```

#
=====

# SECTION 4: UI HELPER COMPONENTS
#
=====

def card(children, style=None):
    base = {'background': COLORS['card'], 'borderRadius': '12px',
            'padding': '20px', 'border': f'1px solid {COLORS["border"]}',
            'boxShadow': '0 2px 12px rgba(0,0,0,0.08)'}
    if style: base.update(style)
    return html.Div(children, style=base)

def stat_box(label, value, color=COLORS['accent1']):
    return html.Div([
        html.Div(label, style={'color': COLORS['muted'], 'fontSize': '11px',
                                'textTransform': 'uppercase', 'letterSpacing': '1px',
                                'marginBottom': '6px'}),
        html.Div(value, style={'color': color, 'fontSize': '22px',
                                'fontWeight': '700'}),
    ], style={'background': COLORS['card2'], 'borderRadius': '10px',
            'padding': '16px 18px', 'borderLeft': f'3px solid {color}',
            'flex': '1', 'minWidth': '160px'})

def apply_layout(fig, **kwargs):
    layout = dict(PLOT_LAYOUT)
    layout.update(kwargs)
    fig.update_layout(**layout)
    return fig

def section_title(text, color=COLORS['accent1']):
    return html.H4(text, style={'color': color, 'marginTop': 0,
                                'marginBottom': '14px'})

```

```

#
=====
=====

```

```

# SECTION 5: PRE-COMPUTE GLOBAL KPIs (runs once at startup)
#

=====

_snap_global = df.groupby('location').agg(
    total_cases=('total_cases', 'max'),
    total_deaths=('total_deaths', 'max'),
    total_vaccinations=('total_vaccinations', 'max'),
).reset_index()

_total_cases = _snap_global['total_cases'].sum()
_total_deaths = _snap_global['total_deaths'].sum()
_total_vacc = _snap_global['total_vaccinations'].sum()
_cfr = (_total_deaths / _total_cases * 100) if _total_cases else 0
_n_countries = df['location'].nunique()

# Build KPI strip shown at top of every tab
_KPI_CHILDREN = [
    stat_box('Total Cases',    f'{_total_cases/1e6:.1f}M', COLORS['accent3']),
    stat_box('Total Deaths',  f'{_total_deaths/1e6:.2f}M', COLORS['accent2']),
    stat_box('Vaccinations',   f'{_total_vacc/1e9:.2f}B',  COLORS['accent4']),
    stat_box('Case Fatality Rate', f'{_cfr:.2f}%',        COLORS['accent5']),
    stat_box('Countries Tracked', str(_n_countries),      COLORS['accent1']),
    stat_box('Data Period',    '2020 - 2022',            COLORS['muted']),
]

#

=====

# SECTION 6: APP LAYOUT & TAB ROUTER
#

=====

app = Dash(__name__, suppress_callback_exceptions=True)
app.title = 'COVID-19 Statistical Analysis Dashboard'

app.layout = html.Div(style={
    'background': COLORS['bg'], 'minHeight': '100vh',

```



```

'color': COLORS['text'],
}, children=[
# _____ HEADER

html.Div([
  html.Div([
    html.Span('COVID-19', style={'color': COLORS['accent1'],
      'fontSize': '24px', 'fontWeight': '800'}),
    html.Span(' Statistical Analysis Dashboard',
      style={'color': COLORS['text'], 'fontSize': '24px'}),
  ]),
  html.Div('Global Pandemic Data 2020-2022 · Stat-isfaction Team',
    style={'color': COLORS['muted'], 'fontSize': '12px'}),
], style={'background': COLORS['card'], 'padding': '20px 32px',
  'borderBottom': '2px solid {COLORS["accent1"]}'}),
# — KPI STRIP —
html.Div(_KPI_CHILDREN, style={'display': 'flex', 'gap': '12px',
  'padding': '18px 24px', 'flexWrap': 'wrap'}),
# _____ TABS

dcc.Tabs(id='tabs', value='overview', children=[
  dcc.Tab(label='Overview', value='overview'),
  dcc.Tab(label='Time Series', value='timeseries'),
  dcc.Tab(label='Descriptive', value='descriptive'),
  dcc.Tab(label='Probability', value='probability'),
  dcc.Tab(label='Regression', value='regression'),
  dcc.Tab(label='Compare', value='compare'),
]),
html.Div(id='tab-content', style={'padding': '20px 24px'}),
])

@app.callback(Output('tab-content', 'children'), Input('tabs', 'value'))
def render_tab(tab):
  if tab == 'overview': return layout_overview()
  if tab == 'timeseries': return layout_timeseries()
  if tab == 'descriptive': return layout_descriptive()
  if tab == 'probability': return layout_probability()
  if tab == 'regression': return layout_regression()

```

```

if tab == 'compare':    return layout_compare()
return html.Div('Select a tab')

```

```

#

```

```

# SECTION 7: TAB 1: GLOBAL OVERVIEW
#

```

```

def layout_overview():

```

```

    # Per-country snapshot: max cumulative values

```

```

    snap = df.groupby('location').agg(
        iso_code=('iso_code','first'),
        total_cases=('total_cases','max'),
        total_deaths=('total_deaths','max'),
        continent=('continent','first'),
    ).reset_index()

```

```

    # World choropleth map, Blues colour scale

```

```

    fig_map = px.choropleth(
        snap, locations='iso_code', color='total_cases',
        hover_name='location', color_continuous_scale='Blues',
        title='Total Confirmed Cases by Country',
    )
    apply_layout(fig_map, coloraxis_colorbar=dict(title='Cases'))

```

```

    # Top-15 horizontal bar chart

```

```

    top15 = snap.nlargest(15, 'total_cases')
    fig_bar = px.bar(top15, x='total_cases', y='location',
        orientation='h', color='total_cases',
        color_continuous_scale='Blues',
        title='Top 15 Countries by Total Cases')
    apply_layout(fig_bar, yaxis_categoryorder='total ascending')

```

```

    # Continent share donut

```

```

    cont_cases = df[df['continent'].notna()].groupby('continent')
        ['new_cases'].sum().reset_index()

```

```

fig_pie = px.pie(cont_cases, values='new_cases', names='continent',
                 title='Share of Cases by Continent',
                 color='continent', color_discrete_map=CONT_COLORS, hole=0.45)

# Log-log deaths vs cases scatter
snap2 = snap[(snap['total_cases'] > 1000) & snap['continent'].notna()]
fig_sc = px.scatter(snap2, x='total_cases', y='total_deaths',
                   color='continent', hover_name='location',
                   color_discrete_map=CONT_COLORS,
                   title='Total Deaths vs Total Cases (log scale)',
                   log_x=True, log_y=True,
                   size='total_deaths', size_max=30)
apply_layout(fig_sc)
return html.Div([card(dcc.Graph(figure=fig_map)), ...])

```

```
#
```

```
# SECTION 8: TAB 2: TIME SERIES
```

```
#
```

```

@app.callback(
    Output('ts-graph','figure'), Output('ts-rolling','figure'),
    Input('ts-countries','value'), Input('ts-metric','value'),
)
def update_ts(selected_countries, metric):
    if not selected_countries:
        return go.Figure(), go.Figure()
    sub = df[df['location'].isin(selected_countries)]
    fig = go.Figure() # Raw daily
    fig2 = go.Figure() # 7-day rolling average
    for i, loc in enumerate(selected_countries):
        d = sub[sub['location'] == loc].copy()
        color = clr_list[i % len(clr_list)]
        # Raw line
        fig.add_trace(go.Scatter(
            x=d['date'], y=d[metric], name=loc,

```

```

        mode='lines', line=dict(color=color, width=1.5)))
    # 7-day rolling mean (min_periods=1 avoids NaN at start)
    d['rolling'] = d[metric].rolling(7, min_periods=1).mean()
    fig2.add_trace(go.Scatter(
        x=d['date'], y=d['rolling'], name=loc,
        mode='lines', line=dict(color=color, width=2)))
    apply_layout(fig, title=f'{metric} Over Time', hovermode='x unified')
    apply_layout(fig2, title=f'7-Day Rolling Mean - {metric}',
        hovermode='x unified')
    return fig, fig2

```

```
#
```

```
# SECTION 9: TAB 3: DESCRIPTIVE STATISTICS
```

```
#
```

```
def layout_descriptive():
```

```

    numeric_cols = ['new_cases', 'total_cases', 'new_deaths', 'total_deaths',
        'total_vaccinations', 'stringency_index', 'gdp_per_capita',
        'life_expectancy', 'positive_rate', 'hospital_beds_per_thousand']
    snap = df.groupby('location')[numeric_cols].max().reset_index(drop=True)

```

```
    # Full describe() table
```

```

    desc = snap[numeric_cols].describe().T.reset_index()
    desc.columns = ['Variable', 'Count', 'Mean', 'Std Dev', 'Min',
        '25%', 'Median', '75%', 'Max']

```

```
    # 95% Confidence Intervals using Student t
```

```

    ci_vars = ['new_cases', 'total_deaths', 'positive_rate',
        'life_expectancy', 'gdp_per_capita']

```

```
    ci_rows = []
```

```
    for col in ci_vars:
```

```
        vals = snap[col].dropna()
```

```
        n = len(vals)
```

```
        mean = vals.mean()
```

```
        se = stats.sem(vals)      # std / sqrt(n)
```

```

ci = stats.t.interval(0.95, df=n-1, loc=mean, scale=se)
ci_rows.append({
    'Variable': col.replace('_', '').title(),
    'N': n, 'Mean': f'{mean:,.2f}',
    'Std Error': f'{se:,.2f}',
    '95% CI Lower': f'{ci[0]:,.2f}',
    '95% CI Upper': f'{ci[1]:,.2f}',
})

# Box plots, 4 variables side by side
fig_box = make_subplots(rows=1, cols=4,
    subplot_titles=['New Cases', 'New Deaths',
        'Positive Rate', 'Life Expectancy'])
box_cols = ['new_cases', 'new_deaths', 'positive_rate', 'life_expectancy']
for i, col in enumerate(box_cols, 1):
    fig_box.add_trace(
        go.Box(y=snap[col].dropna(), name=col.replace('_', '').title(),
            boxmean='sd'), # Show mean and SD on box
        row=1, col=i)

# Histogram clipped at 95th pctile to manage outlier distortion
clip_val = snap['new_cases'].quantile(0.95)
fig_hist = go.Figure()
fig_hist.add_trace(go.Histogram(
    x=snap['new_cases'].clip(0, clip_val), nbinsx=40,
    marker_color=COLORS['accent1'], opacity=0.8))

```

```
#
```

```
# SECTION 10: TAB 4: PROBABILITY & DISTRIBUTIONS
```

```
#
```

```
def update_prob(var):
    snap = df.groupby('location')[var].max().dropna()
    vals = snap[snap > 0].values
```

```

# Fit Normal distribution (MLE)
mu, sigma = stats.norm.fit(vals)

# Histogram + Normal PDF overlay
x_pdf = np.linspace(vals.min(), vals.max(), 300)
pdf = stats.norm.pdf(x_pdf, mu, sigma)
fig_hist = go.Figure()
fig_hist.add_trace(go.Histogram(x=vals, nbinsx=40,
    histnorm='probability density', name='Observed',
    marker_color=COLORS['accent1'], opacity=0.6))
fig_hist.add_trace(go.Scatter(x=x_pdf, y=pdf,
    mode='lines', name='Normal PDF',
    line=dict(color=COLORS['accent2'], width=2.5)))

# Q-Q Plot
(osm, osr), _ = stats.probplot(vals, dist='norm')
fig_qq = go.Figure()
fig_qq.add_trace(go.Scatter(x=osm, y=osr, mode='markers',
    marker=dict(color=COLORS['accent1'], size=6, opacity=0.7)))
line_y = np.array([osm.min(), osm.max()]) * osr.std() + osr.mean()
fig_qq.add_trace(go.Scatter(x=[osm.min(), osm.max()], y=line_y,
    mode='lines', name='Normal Reference',
    line=dict(color=COLORS['accent2'], dash='dash', width=2)))

# Normality test
if len(vals) <= 5000:
    stat_t, p_val = stats.shapiro(vals[:5000]) # Shapiro-Wilk
else:
    stat_t, p_val = stats.kstest(vals, 'norm', args=(mu, sigma)) # KS

# Descriptive probability metrics
skewness = stats.skew(vals)
kurt = stats.kurtosis(vals)
within_1s = (np.abs(vals-mu) < sigma).mean() * 100
within_2s = (np.abs(vals-mu) < 2*sigma).mean() * 100
pct_above = (vals > mu).mean() * 100
return fig_hist, fig_qq, [...] # Returns info panel children

```

```

# Probability Calculator callback
@app.callback(Output('prob-calc-result','children'),
              Input('prob-calc-btn','n_clicks'),
              State('prob-threshold','value'), State('prob-var','value'),
              prevent_initial_call=True)
def compute_probability(n_clicks, threshold, var):
    snap = df.groupby('location')[var].max().dropna()
    vals = snap[snap > 0].values
    mu, sigma = stats.norm.fit(vals)
    p_above = 1 - stats.norm.cdf(threshold, mu, sigma)
    p_below = stats.norm.cdf(threshold, mu, sigma)
    emp_above = (vals > threshold).mean() # Empirical probability
    n_above = (vals > threshold).sum()
    n_total = len(vals)
    return html.Div([...]) # Display stat boxes with results

```

```

#

```

```

# SECTION 11: TAB 5: REGRESSION ANALYSIS
#

```

```

@app.callback(
    Output('reg-scatter','figure'), Output('reg-summary','children'),
    Output('multi-reg','children'), Output('reg-resid','figure'),
    Input('reg-x','value'), Input('reg-y','value'),
)
def update_regression(x_col, y_col):
    snap = df.groupby('location')[unique_cols].max().reset_index()
    snap = snap.dropna(subset=[x_col, y_col])
    snap = snap[(snap[x_col] > 0) & (snap[y_col] > 0)]
    X_val, Y_val = snap[x_col].values, snap[y_col].values

    # Simple OLS via scipy
    slope, intercept, r_value, p_value, std_err = \
        stats.linregress(X_val, Y_val)

```

```

# Regression line
x_line = np.linspace(X_val.min(), X_val.max(), 200)
y_line = slope * x_line + intercept

# Multiple regression via statsmodels
multi_cols = ['gdp_per_capita', 'life_expectancy',
              'aged_65_older', 'hospital_beds_per_thousand']
snap_m = df.groupby('location')[['total_deaths']+multi_cols]
        .max().reset_index(drop=True)
snap_m = snap_m.dropna().replace([np.inf, -np.inf], np.nan).dropna()
snap_m = snap_m[snap_m['total_deaths'] > 0]
X_m = sm.add_constant(snap_m[multi_cols])
model = sm.OLS(snap_m['total_deaths'], X_m).fit()

# Residuals
y_pred = slope * X_val + intercept
residuals = Y_val - y_pred

return fig_sc, summary, multi_content, fig_resid

# Prediction with 95% Prediction Interval
@app.callback(Output('reg-pred-result', 'children'), ...)
def make_prediction(n_clicks, x_input, x_col, y_col):
    slope, intercept, r, p, se = stats.linregress(X_val, Y_val)
    y_pred = slope * x_input + intercept
    s_e = np.sqrt(np.sum((Y_val - (slope * X_val + intercept))**2) / (n-2))
    se_pred = s_e * np.sqrt(1 + 1/n + (x_input - x_mean)**2 /
                           np.sum((X_val - x_mean)**2))
    t_crit = stats.t.ppf(0.975, df=n-2)
    pi_lower, pi_upper = y_pred - t_crit * se_pred, y_pred + t_crit * se_pred

#
=====
=====
# SECTION 12: TAB 6: COUNTRY COMPARISON
#
=====
=====

```



```

@app.callback(Output('cmp-content','children'),
              Input('cmp-countries','value'))
def update_compare(selected):
    snap = df[df['location'].isin(selected)].groupby('location').agg(
        total_cases=('total_cases','max'),
        total_deaths=('total_deaths','max'),
        total_vaccinations=('total_vaccinations','max'),
        population=('population','max'),
        gdp_per_capita=('gdp_per_capita','max'),
        life_expectancy=('life_expectancy','max'),
        stringency_index=('stringency_index','mean'),
    ).reset_index()

    # Derived metrics
    snap['CFR (%)'] = snap['total_deaths']/snap['total_cases']*100
    snap['Cases/Million'] = snap['total_cases']/snap['population']*1e6
    snap['Deaths/Million'] = snap['total_deaths']/snap['population']*1e6

    # Radar chart, normalise each metric to 0-100
    radar_metrics = ['CFR (%)','Cases/Million','Deaths/Million',
                    'life_expectancy','stringency_index']
    fig_radar = go.Figure()
    for _, row in snap.iterrows():
        vals_r = [row[m] for m in radar_metrics]
        maxes = [snap[m].max() for m in radar_metrics]
        vals_n = [v/mx*100 if mx else 0 for v,mx in zip(vals_r,maxes)]
        fig_radar.add_trace(go.Scatterpolar(
            r=vals_n+[vals_n[0]], theta=radar_metrics+[radar_metrics[0]],
            name=row['location'], mode='lines+markers'))

    # Grouped bar: 3 sub-plots for cases, deaths, vaccinations
    figBars = make_subplots(rows=1, cols=3,
        subplot_titles=['Total Cases','Total Deaths','Total Vaccinations'])
    for j, m in enumerate(['total_cases','total_deaths','total_vaccinations']):
        for i, row in snap.iterrows():
            figBars.add_trace(go.Bar(
                x=[row['location']], y=[row[m]], name=row['location'],
                showlegend=(j==0)), row=1, col=j+1)

```

```
return html.Div([card(dcc.Graph(figure=fig_radar)),
                  card(dcc.Graph(figure=fig_bars)),
                  card([table])])
```

```
#
```

```
# SECTION 13: RUN SERVER
```

```
#
```

```
if __name__ == '__main__':
    app.run(debug=False, host='0.0.0.0', port=8050)
    # Access at: http://127.0.0.1:8050/
```

6. Conclusion

This project demonstrates the end-to-end application of probability and statistics to one of the most consequential real-world datasets of the 21st century. The COVID-19 Statistical Analysis Dashboard (developed entirely in Python using the Dash/Plotly ecosystem alongside pandas, NumPy, SciPy, and Statsmodels) successfully transforms a 200,000-row, 67-column epidemiological dataset into a fully interactive, multi-tab analytical platform accessible via a standard web browser.

From a statistical methodology standpoint, the project covers the complete pipeline prescribed in the Probability & Statistics curriculum:

Descriptive Statistics:

The dashboard computes and renders mean, median, standard deviation, quartiles, and range for ten key pandemic metrics. Box plots and histograms reveal the extreme right-skew characteristic of pandemic case distributions (a critical insight that cannot be gleaned from raw totals alone).

Confidence Intervals:

95% Student's t-intervals are computed for five variables. The results quantify the uncertainty inherent in using a finite sample of ~221 countries to estimate global population means. Life expectancy CIs are narrow (± 1.07 years) reflecting low inter-country variability; new cases CIs are very wide ($\pm 875,656$) reflecting enormous heterogeneity.

Probability & Distribution Fitting:

Shapiro-Wilk and Kolmogorov-Smirnov normality tests confirm that COVID-19 case and death distributions are significantly non-normal ($p < 0.001$). Log-Normal distributions provide substantially better fits, consistent with the multiplicative growth dynamics of infectious disease. Life expectancy is the only variable that approaches normality ($p \approx 0.04$). The interactive probability calculator makes the concept of $P(X > T)$ directly explorable by comparing model-based and empirical estimates.

Regression Analysis:

Simple OLS regression reveals moderate positive correlations between GDP per capita and total reported deaths ($R^2 \approx 0.27$, $p < 0.001$) and between life expectancy and deaths, relationships driven by the higher testing and reporting capacity in wealthier nations rather than by genuine causal mortality risk. The multiple regression model ($R^2 \approx 0.35$) shows that GDP and life expectancy are independently significant predictors even after controlling for age structure and

hospital capacity. Residual analysis reveals heteroscedasticity, motivating log-transformation for future work. The prediction interval tool correctly conveys the wide uncertainty of country-level predictions.

Country Comparison:

The radar chart and comparison table highlight a critical insight: raw case and death counts are dominated by population size and testing capacity. Per-million metrics reveal a more nuanced picture, e.g., Germany's 435,000 cases/million versus Pakistan's 7,100 cases/million is primarily a testing-intensity difference, not a true epidemiological $60\times$ difference in infection rates.

In terms of software engineering, the Dash callback architecture cleanly separates UI definition from analytical logic, enabling real-time interactivity without page reloads. Pre-computing global KPIs at startup and using per-callback grouped aggregation ensures the application remains responsive even on a moderate-spec machine. The modular tab structure makes the codebase straightforward to extend, with potential enhancements including ARIMA/Prophet time-series forecasting, Bayesian credible intervals, spatial autocorrelation analysis (Moran's I), or excess-mortality computation.

Overall, the project achieves its stated objectives in full: it delivers a statistically rigorous, visually polished, and genuinely interactive analytical tool that makes the mathematics of probability and statistics tangible through one of the most important datasets of our generation. The team (Stat-isfaction) is confident that the dashboard serves as both a meaningful academic deliverable and a reusable foundation for future epidemiological data science work.

7. References

[1] Our World in Data:

Mathieu, E., Ritchie, H., Ortiz-Ospina, E. et al. (2020). A global database of COVID-19 vaccinations. Nature Human Behaviour.

<https://github.com/owid/covid-19-data>

[2] Plotly & Dash:

Plotly Technologies Inc. (2015). Collaborative data science. Plotly. Montreal, QC.

<https://plot.ly>

<https://dash.plotly.com>

[3] SciPy:

Virtanen, P. et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nature Methods, 17, 261–272.

<https://scipy.org>

[4] Statsmodels:

Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with Python. Proceedings of the 9th Python in Science Conference.

<https://www.statsmodels.org>

[5] pandas:

McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 445, 51–56.

<https://pandas.pydata.org>

[6] Oxford Stringency Index:

Hale, T. et al. (2021). A global panel database of pandemic policies. Nature Human Behaviour, 5, 529–538.

<https://www.bsg.ox.ac.uk/research/covid-19-government-response-tracker>

[7] WHO COVID-19 Dashboard:

World Health Organization. (2023). WHO Coronavirus (COVID-19) Dashboard.

<https://covid19.who.int>